



Proyecto Especial

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v3.0.5

WEIRD FLEX BUT OK

Materia:

(2Q2024) 72.39 Autómatas, Teoría de Lenguajes y Compiladores

Profesores:

Ana María Arias Roig

Mario Agustín Golmar

Integrantes:

Lonegro Gurfinkel, Lucas 63.738 - llonegrourfinkel@itba.edu.ar

Noceda, Tobías Martín 63.422 - tnoceda@itba.edu.ar

Smirnoff, Sergio - 62.256 - ssmirnoff@itba.edu.ar

Torlaschi, Patrick Luca 62.273 - ptorlaschi@itba.edu.ar

1. Introducción	3
2. Modelo Computacional	3
2.1. Dominio	3
2.2. Lenguaje	3
3. Implementación	4
3.1. Frontend	4
3.2. Backend	5
3.3. Adicionales	5
3.4. Dificultades Encontradas	6
4. Futuras Extensiones	7
4.1. Negación de una Expresión Regular	7
4.2. Contextos	7
4.3. Mayor Cubrimiento de Java	7
4.4. Algoritmo de Hopcroft para Minimización de Autómatas	7
4.5. Resolución de Ciclos en Cálculo de Clausuras Lambda	7
5. Conclusiones	8
6. Referencias	8
7. Bibliografía	8
8. Apéndice	8

1. Introducción

En este trabajo se centra en el desarrollo de un compilador capaz de generar analizadores léxicos en el lenguaje Java. El objetivo principal de este es generar a partir de unas reglas semánticas definidas en nuestro lenguaje con ciertos criterios se permita generar un código en java el cual permita generar tokens dependiendo de la entrada para poder luego ser consumidos en un analizador sintáctico.

Para la implementación del compilador, se utilizó el lenguaje de programación C junto con las herramientas Flex, analizador léxico, y Bison, analizador sintáctico. Este informe detalla el proceso de desarrollo del lenguaje de reglas y la conversión a su equivalente en lenguaje Java, explicando complicaciones y sus respectivas soluciones.

2. Modelo Computacional

2.1. Dominio

Se desarrolló un lenguaje de dominio específico (DSL) que permite la creación de un conjunto de reglas son utilizadas para generar tokens dependiendo de la entrada. Luego de ser analizadas las reglas se puede generar un código en Java capaz de generar tokens que se adecuen a las reglas previamente definidas.

El lenguaje definido se elaboró para que cuente con las siguientes características: fácil de usar, cuenta con un único lugar de definición de funciones y variables, genera un único token por cada estado y cada token es inmodificable desde el exterior del programa. El lenguaje define las reglas que serán utilizadas para definir qué tipo de token se emitirá en cada caso, dependiendo del uso que se le quiera dar al analizador léxico.

En última instancia, se emite un código en Java teniendo en cuenta las reglas creadas anteriormente, pudiendo así, analizar léxicamente la entrada que se le dé al programa.

2.2. Lenguaje

El lenguaje desarrollado ofrece las siguientes características:

2.2.1. Definición de expresiones regulares.

El lenguaje reconoce expresiones de la forma:

$[A-Z][A-Z0-9]^*$

2.2.2. Permitir definir macros, funciones y reglas en el mismo sector.

```

Definición de macros:
Regex digit      [0-9];
Regex letter     [a-zA-Z];
Definición de funciones:
{letter}[[{letter}{digit}]* -> {
    Token tok = IDENT.setStringContent(var.getLexeme());
    return tok;
};
Definición de reglas:
“abrir” -> OPEN;
    
```

2.2.3. Permitir definir reglas a partir de otras reglas (macros) previamente definidas.

Definición de macros: Regex <nombre> [...] y luego utilizarlas de la forma: {<nombre>}.

2.2.4. Elaborar los tokens antes de entrar al bison.

Se elaboran y envían los tokens correspondientes a las flex actions las cuales son encargadas de enviarlas a bison.

2.2.5. Utilizar variables locales de contexto para cada token haciendo que no se puedan modificar estas variables desde otros entornos.

Las variables que se generan y los tokens que se emiten no pueden ser modificados luego de ser emitidos. Una vez lanzados por el programa de java no se puede modificar su estado globalmente.

2.2.6. Utilizar “gramática” y estilo similar a java para facilitar su transición.

Se utiliza java dentro de las definiciones de las funciones, lo cual facilita al usuario entender cómo y cuándo se va a ejecutar ese código y componerlo de la forma que se requiera.

2.2.7. Retornar un archivo con extensión .java.

El retorno del compilador es un programa en Java que se encarga de tokenizar la entrada que se le dé.

2.2.8. Permitir comentarios de líneas múltiples con /* */ de forma que se ignore todo lo que hay entre las /

```

/*
 * I am a comment.
 */
    
```

3. Implementación

3.1. Frontend

En esta sección, se detalla el desarrollo del frontend del compilador el cual contiene las etapas de definición de estructuras que se van a utilizar, construcción de las reglas del analizador léxico y la formación del analizador sintáctico para comprender y realizar las tareas explicadas anteriormente.

Se tienen las definiciones de las estructuras elegidas para representar cada componente del lenguaje. Estas se pueden dividir en 2 grandes grupos, las básicas que analizan las composiciones de las reglas, funciones y macros que se definen en el lenguaje y, luego, las estructuras que se encargan de interpretar el lenguaje Java que está contenido dentro de las funciones. Para las funciones básicas referirse al apéndice (ver 8.1).

Por otro lado se encuentra el analizador léxico. Se define el conjunto de expresiones reservadas, el RETURN, operadores de comparación ($\leq$ | $<$ | $>$ | etc.), entre otros. Luego la aceptación de macros prefijadas por la palabra reservada Regex o regex, manejo de comentarios multilínea y, por último, los manejos de expresiones regulares básicas, tokens reservados, y funciones.

Por último, el analizador sintáctico Bison. La gramática definida puede dividirse en dos secciones, la encargada de manejar el contenido de las reglas que se van definiendo en el archivo de entrada y la que analiza bloques de código de un sublenguaje de Java. En esta segunda sección se incluyen las herramientas básicas del lenguaje.

3.2. Backend

A continuación, se describe el desarrollo del backend del compilador, el cual contiene 3 partes principales en su flujo. Estas son el Generator (code-generation), el WeirdFlexButOk (domain-specific) y por último el automaton.

Para comenzar, en el WeirdFlexButOk se recorre el AST y se forma una tabla en donde se relacionan los lexemas con el contenido de los mismos, para saber qué acción se debe realizar ante su entrada. Esta tabla se entrega al constructor del autómata.

Luego, la carpeta automaton define funciones y tipos de datos abstractos para la generación y manipulación de un autómata. Se genera recursivamente en base a el AST un autómata finito no determinístico lambda (AFND- λ). Este es utilizado para generar un autómata finito no determinístico (AFND) que a su vez genera un autómata finito determinístico (AFD) a partir del cual se obtiene un equivalente mínimo. Este último AFD se utiliza para la generación del código del autómata en el archivo de salida.

Por último, en el archivo Generator.c se recorren los estados del autómata, creado previamente, y se traduce a código interpretable/ ejecutable de java. Esto se hace mediante llamadas recursivas para procesar desde lo más simple (Literales/ Variables) a lo más complejo (Blocks/ Expresiones/ Ciclos). El código traducido se escribe en el archivo de salida "automaton.java" el cual presenta un autómata funcional capaz de generar tokens en base a un input.

3.3. Adicionales

Cálculo de Clausuras Lambda

Tras completar la implementación del backend para la versión inicial del lenguaje, se notó una clara dificultad en la construcción del autómata respecto a las clausuras de Kleene. Para facilitar esta construcción en extensiones planeadas, se decidió implementar el uso y la resolución de transiciones lambda en la librería de autómatas.

Extensión de Clausuras de Kleene en Expresiones Regulares

La implementación inicial del lenguaje soporta clausuras de Kleene únicamente para subconjuntos muy acotados de las expresiones regulares. Se extendió el lenguaje para incluir en la definición de expresión regular la clausura de una expresión regular:

Si r es una expresión regular
Entonces (r) , $(r)^*$, $(r)^+$ son
expresiones regulares

Luego, se hizo uso del soporte de transiciones lambda para implementar este manejo en el backend.

Operador de Disyunción en Expresiones Regulares

La implementación inicial del lenguaje soporta únicamente disyunciones de símbolos individuales. Se extendió el lenguaje para incluir en la definición de expresión regular la disyunción de dos expresiones regulares:

Si r_1 , r_2 son expresiones regulares
Entonces $r_1|r_2$ es una expresión
regular

Similarmente, se hizo uso del soporte de transiciones lambda para implementar este manejo en el backend.

Minimización del Autómata

Se implementó el algoritmo de Moore para minimización de autómatas¹. Mientras que esto no produce reducciones en la complejidad temporal del programa Java resultante, sí tiene dos ventajas principales: Primero, la definición del autómata resultante suele ser más acotada, ocupando menor espacio en memoria. Luego, al ser obtenida la forma canónica del autómata, es más fácil corroborar el correcto funcionamiento del programa, además de que el resultado es más legible.

¹ Algoritmo de Moore, apéndice 8.2

3.4. Dificultades Encontradas

Eficiencia

El backend de este proyecto requirió la implementación de tres operaciones costosas: La conversión de un AFND- λ a un AFND equivalente, la conversión de un AFND a un AFD equivalente y la minimización de un AFD. Las implementaciones iniciales de estas últimas dos operaciones resultaron no escalar a autómatas con mayores cantidades de estados.

Se utilizó la herramienta GNU *gprof* para analizar los puntos de ineficiencias que podrían ser resueltos con una mejor selección de estructuras de datos. Luego, para la conversión de AFND a AFD se programó un tipo de dato abstracto para hashing de estados de un autómata; para la minimización de un AFD se utilizó una tabla de búsqueda que relaciona estados con sus particiones.

Parsing de Java

Como el output del compilador es código Java, es necesario recibir en ciertas partes del lenguaje código Java creado por el usuario. El lenguaje de programación Java comprende una amplia muestra de construcciones - intentar replicarlas en una gramática con Bison como herramienta no solo sería impráctico, sino que escapa tanto los objetivos como los alcances del proyecto. Se resolvió por acotar el lenguaje aceptado a un subconjunto de Java; se produjo una gramática reducida basada cercanamente en la especificación de Java 17².

4. Futuras Extensiones

4.1. Negación de una Expresión Regular

Una funcionalidad ofrecida por la mayoría de los lenguajes que aceptan regex aceptan la negación de un símbolo para así aceptar todo símbolo excepto aquel que se especifica. Esta es una funcionalidad que se omitió para eliminar algo de complejidad en la construcción del lenguaje pero que podría añadirse en un futuro dado que robustece aún más el lenguaje

4.2. Contextos

Actualmente se carece de contextos que permitan dar prioridad a una regla sobre otras en un contexto determinado. Esto es algo que es muy útil para la aceptación de comentarios multilínea por ejemplo tal y como se usó en este mismo proyecto. Esto le da más versatilidad, flexibilidad y legibilidad al desarrollo del analizador léxico.

4.3. Mayor Cubrimiento de Java

Actualmente, se alcanza un recubrimiento parcial de muchas de las funciones más utilizadas de Java como puede ser FOR, WHILE e IF. Para un futuro se podría reconocer más funciones del lenguaje para darle al compilador mayor utilidad y reconocimiento de lenguajes; entre las más relevantes, lectura y escritura a arreglos, operadores de parametrización y ciclos FOR-EACH.

² Extraído del libro *Formal Grammar for Java*. (Ref 6.2)

4.4. Algoritmo de Hopcroft para Minimización de Autómatas

Actualmente, la función utilizada para generar el autómata finito mínimo equivalente para un autómata finito determinístico es una implementación del algoritmo de Moore, cuya complejidad temporal en el peor caso es $O(n^2s)^3$. Mientras que esta solución alcanza esta complejidad mínima, existen algoritmos con menores complejidades, como el algoritmo de Hopcroft, con un peor caso de $O(n\log(n))^4$.

4.5. Resolución de Ciclos en Cálculo de Clausuras Lambda

Se analizó los casos de uso de la librería de manejo de autómatas y se concluyó que esta función de la librería no requería optimización significativa; tanto la generación de un AFD equivalente a un AFND como la minimización de un AFD son órdenes de magnitud más costosos temporalmente que este cálculo. Por esta razón, se usó una implementación naïve para el cálculo de clausuras lambda para cada estado de un AFND- λ .

Existen multitud de soluciones alternativas mejores, una de estas se define en dos pasos: Se considera un grafo cuyos vértices son los estados del autómata y sus aristas las transiciones lambda. En primer lugar, todos los estados que se encuentran en un ciclo dirigido se consideran equivalentes, sus clausuras lambda serán iguales, por lo que sus vértices son combinados en uno que conserva el conjunto de todos sus ejes. Una vez que el autómata no contiene ciclos dirigidos, se pueden ejecutar búsquedas a profundidad en subgrafos dirigidos conexos hasta haber visitado todos los vértices, la clausura lambda correspondiente a cada vértice es el conjunto de estados representados por los vértices del subárbol en el que es raíz en la búsqueda ejecutada. Este recorrido permite una optimización del cálculo de clausuras mediante programación dinámica: la clausura de los estados representados por un vértice es la unión de las clausuras de sus vértices adyacentes y los estados que este mismo vértice representa.

³ Como se explica en Gedanken-Experiments on Sequential Machines (ref 6.1)

⁴ E. Hopcroft. *An $n \log n$ algorithm for minimizing in a finite automaton*. In Proc. International Symposium of Theory of Machines and Computations, pages 189–196. Academic Press, 1971.

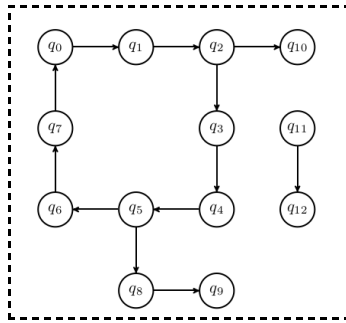


Figura 1.1: Grafo de transiciones lambda

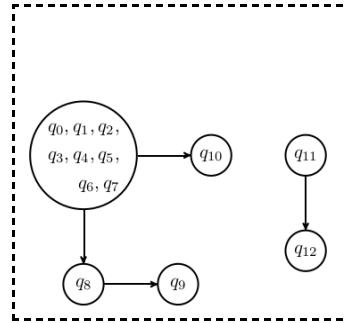


Figura 1.2: Combinación de estados en un ciclo dirigido,

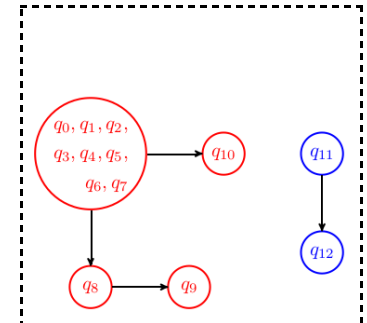


Figura 1.3: Subgrafos dirigidos conexos

$$\begin{aligned} \lambda_{q_j} &= \{q_0; q_1; q_2; q_3; q_4; q_5; q_6; q_7; q_8; q_9; q_{10}\}, q_j \in \{q_0; q_1; q_2; q_3; q_4; q_5; q_6; q_7\} \\ \lambda_{q_8} &= \{q_8; q_9\} \\ \lambda_{q_9} &= \{q_9\} \\ \lambda_{q_{10}} &= \{q_{10}\} \\ \lambda_{q_{11}} &= \{q_{11}; q_{12}\} \\ \lambda_{q_{12}} &= \{q_{12}\} \end{aligned}$$

Figura 1.4: Cálculo de clausuras lambda

5. Conclusiones

En conclusión, el proyecto reforzó los conocimientos sobre análisis léxico y sintáctico, aprendiendo en profundidad el funcionamiento de un compilador y cada uno de sus componentes. Ayudó a comprender las debilidades que tienen los sistemas utilizados actualmente y cómo optimizarlos para lograr el mejor rendimiento de cada uno.

En conjunto, esta implementación contribuye a la comprensión y práctica en el diseño de compiladores, además de servir como base para proyectos futuros relacionados con la generación automática de herramientas para el análisis del lenguaje.

El proceso no estuvo exento de desafíos, como la definición precisa del lenguaje de reglas y la traducción eficiente al código en Java, pero cada complicación fue abordada con soluciones adaptadas al contexto del proyecto. Este trabajo no sólo valida el uso de herramientas clásicas de construcción de compiladores, sino que también abre la puerta para extender este enfoque a otros lenguajes de programación o incluir reglas semánticas más complejas, enriqueciendo el ecosistema del desarrollo de compiladores y herramientas de análisis léxico.

6. Referencias

- 6.1. Moore, E. F. (1956). Gedanken-Experiments on Sequential Machines. In C. E. Shannon & J. McCarthy (Eds.), *Automata Studies* (pp. 129–153). Princeton University Press.
- 6.2. Jim Alves-Foss and Deborah Frincke, January 1999. Formal Grammar for Java. Recuperado el 24 de noviembre de 2024, de https://www.researchgate.net/publication/221552891_Forma_Grammar_for_Java

7. Bibliografía

- 7.1. Edition, J. S. E. 17. (s/f). The java language specification. Oracle.com. Recuperado el 13 de octubre de 2024, de <https://docs.oracle.com/javase/specs/jls/se17/jls17.pdf>

8. Apéndice

- 8.1. Las estructuras definidas en Bison:

<pre> struct Program { Ruleset *ruleset; }; </pre>	<pre> struct Ruleset { Rule *rule; Ruleset *ruleset; }; </pre>
<pre> struct Regexes { Regex_class *regexClass; Regexes *regexes; }; </pre>	<pre> struct Rule { union { struct { char *our_regex_id; Regexes *regexes; }; Lexeme_precursor *lexeme; struct { Lexeme_precursor *lex; Action *action; }; }; Rule_type type; }; </pre>
<pre> typedef struct Regex_class { union { struct { Symbol *startSymbol; Symbol *endSymbol; }; struct </pre>	<pre> struct Lexeme_precursor { Lexeme *lexeme; Lexeme_precursor *lexeme_precursor; Lexeme_precursor_type precursor_type; Lexeme_chain_type chain_type; }; </pre>

<pre> { char *varName; Closure *closure; }; Symbol *symbol; }; Regex_class_type type; } Regex_class;</pre>	
<pre> struct Type { Token stuff; };</pre>	<pre> struct Lexeme { union { Regexes *regexes; char *our_regex_id; char *string; Lexeme_precursor *precursor; }; Closure *closure; Lexeme_type type; };</pre>
<pre> struct Symbol { char *symbol_tok; };</pre>	<pre> struct Closure { Token closure; };</pre>
<pre> struct Block { union { struct { Statement *statement; Block *block; }; Expression *expression; }; Block_type type; };</pre>	<pre> struct Action { union { char *varName; Block *block; }; Action_type type; };</pre>

8.2. Algoritmo de Moore para minimización de autómatas deterministas:

Entrada: $A = (Q, \Sigma, \delta, q_0, F)$

Salida: $A' = (Q', \Sigma, \delta', q_0', F')$ equivalente a A con mínima cantidad de estados.

1. Eliminar estados inaccesibles desde q_0 .
2. Construir un conjunto cociente Q/E .
3. $A' = (Q', \Sigma, \delta', q_0', F')$ donde:

$$Q' = Q / E$$

q_0' es el elemento de Q / E tal que q_0 pertenece a q_0' (la clase donde está q_0).

$F' = \{ s / s \text{ pertenece a } Q / E \text{ y existe } p \text{ perteneciente a } S \text{ tal que } p \text{ pertenece a } F \}$

$\delta'(s_i, a) = s_j$ si solo si existe p perteneciente a s_i y existe q perteneciente a $s_j / \delta(p, a) = q$